
Daftar Isi

Bab 1: Apa Itu KARSA?

- 1.1 Mengapa Bahasa Pemrograman Indonesia?
- 1.2 Filosofi Desain KARSA
- 1.3 Cara Kerja Pipeline KARSA
- 1.4 Instalasi dan Menjalankan KARSA

Bab 2: Variabel dan Data

- 2.1 data — Variabel Reaktif
- 2.2 tetap — Konstanta
- 2.3 ubah — Variabel Mutable Non-Reaktif
- 2.4 Perbandingan Tiga Jenis Variabel
- 2.5 Type Hint

Bab 3: Tipe Data

- 3.1 Angka
- 3.2 Teks
- 3.3 Benar-Salah (Boolean)
- 3.4 Array
- 3.5 Objek

Bab 4: Operator

- 4.1 Operator Aritmatika
- 4.2 Operator Perbandingan
- 4.3 Operator Logika

Bab 5: Output — tampilkan

- 5.1 Tampilkan Dasar
- 5.2 Mode Tampilkan

Bab 6: Proyek Mini — Kalkulator Sederhana

Latihan Soal

Solusi Latihan

Lampiran: Referensi Cepat KARSA vs JS

Bab 1: Apa Itu KARSA?

KARSA adalah bahasa pemrograman yang dirancang khusus untuk pengembangan web menggunakan kata kunci berbahasa Indonesia. Dibangun di atas ekosistem JavaScript, KARSA memungkinkan pemrogram Indonesia menulis kode dalam bahasa mereka sendiri sambil tetap menghasilkan output JavaScript yang kompatibel dengan browser modern. Nama "KARSA" diambil dari bahasa Jawa yang berarti "niat" atau "kemauan", mencerminkan semangat untuk menciptakan solusi teknologi yang aksesibel bagi masyarakat Indonesia.

1.1 Mengapa Bahasa Pemrograman Indonesia?

Bagi sebagian besar orang Indonesia, belajar pemrograman berarti harus menguasai bahasa Inggris terlebih dahulu. Keyword seperti **if**, **else**, **function**, dan **return** bukanlah kata-kata yang digunakan sehari-hari. Hal ini menciptakan hambatan kognitif tambahan: pemula harus menerjemahkan konsep logika ke dalam bahasa yang tidak mereka kenali, sekaligus memahami konsep pemrograman itu sendiri. KARSA menghilangkan hambatan ini dengan menyediakan kata kunci dalam bahasa Indonesia yang secara intuitif mudah dipahami, sehingga pemula dapat fokus pada logika pemrograman tanpa harus bergulat dengan terjemahan bahasa.

Penelitian di bidang pendidikan komputer menunjukkan bahwa penggunaan bahasa ibu dalam pembelajaran pemrograman secara signifikan meningkatkan kecepatan pemahaman dan menurunkan tingkat kecemasan belajar. Ketika seorang siswa membaca **jika x lebih dari 10** dibandingkan **if (x > 10)**, otak tidak perlu melakukan langkah penerjemahan tambahan. Makna kode menjadi langsung dan transparan. KARSA hadir sebagai jembatan antara kemudahan pemahaman berbahasa Indonesia dan kekuatan penuh ekosistem JavaScript di web.

1.2 Filosofi Desain KARSA

KARSA dibangun dengan empat prinsip desain utama yang memandu setiap keputusan dalam pembuatan bahasa ini. Pertama, **Keterbacaan di Atas Keringkasan** — kode KARSA harus dapat dibaca dan dipahami seperti membaca kalimat bahasa Indonesia. Contohnya, **simpan 20 ke x** lebih mudah dipahami daripada **x = 20** bagi pemula yang baru mengenal konsep assignment. Kedua, **Keselamatan di Atas Kebebasan** — KARSA memilih untuk melindungi pemrogram dari kesalahan umum melalui sistem error code yang komprehensif (E1xxx hingga E5xxx) dan validasi di setiap tahap pipeline, daripada memberikan kebebasan penuh yang bisa berujung pada bug yang sulit dilacak.

Ketiga, **Reaktivitas Bawaan** — KARSA mengadopsi model reaktivitas berbasis Proxy (mirip dengan Vue.js dan SolidJS) sebagai paradigma utama, bukan tambahan opsional. Variabel **data** secara otomatis reaktif, artinya setiap perubahan nilai akan memicu pembaruan tampilan tanpa perlu kode tambahan. Keempat, **Kompatibilitas Web Penuh** — output KARSA adalah JavaScript vanilla yang berjalan di semua browser tanpa dependensi eksternal. Tidak ada runtime khusus yang perlu diinstal; cukup sertakan file hasil kompilasi di halaman HTML dan aplikasi akan berjalan.

1.3 Cara Kerja Pipeline KARSA

Ketika kamu menulis kode KARSA dan menjalankannya, kode tersebut melewati lima tahap pemrosesan yang disebut "pipeline". Setiap tahap memiliki tanggung jawab spesifik dan sistem error-nya sendiri. Pemahaman tentang pipeline ini akan membantumu memahami darimana pesan error berasal dan bagaimana memperbaikinya dengan tepat.

KARSA	JavaScript	Keterangan
Lexer (E1xxx)	Tokenization	Memecah kode sumber menjadi token (kata kunci, identifier, angka, dll.)
Parser (E2xxx)	AST Generation	Membangun Abstract Syntax Tree dari token yang dihasilkan Lexer
Resolver (E3xxx)	Semantic Binding	Mengikat identifier ke deklarasi, deteksi duplikat, dan shadowing
Analyzer (E4xxx)	Validation	Validasi semantik: tipe, reaktivitas, kontrol alur, lifecycle
Compiler (E5xxx)	Code Generation	Menurunkan AST menjadi JavaScript vanilla yang siap eksekusi

Misalnya, jika kamu menulis **data x = 10**, Lexer akan menghasilkan token [data, x, =, 10]. Parser akan membentuk node `DataDeclaration`. Resolver akan mendaftarkan "x" sebagai simbol reaktif. Analyzer akan memvalidasi bahwa nilai awal sesuai type hint (jika ada). Compiler akan menghasilkan **const x = __createReactive(10)**. Setiap tahap dapat menghasilkan error dengan kode spesifik yang membantu proses debugging.

1.4 Instalasi dan Menjalankan KARSA

KARSA dapat digunakan di lingkungan Node.js maupun browser. Untuk instalasi, kamu bisa meng-clone repositori resmi KARSA dari GitHub dan mengintegrasikannya ke dalam proyek webmu. Berikut adalah langkah-langkah dasar untuk memulai:

Terminal

```
# Clone repositori KARSA
git clone https://github.com/dazep01/karsa.git
cd karsa

# Instal dependensi (jika ada)
npm install

# Jalankan di Node.js
node -e "const Karsa = require('./engine/karsa');
const result = Karsa.compile('data x = 10');
console.log(result.js);"
```

Untuk penggunaan di browser, cukup sertakan file engine KARSA dan gunakan tag `<script type="text/karsa">` untuk menulis kode KARSA secara inline. KARSA akan secara otomatis mencari dan mengkompilasi semua tag tersebut saat halaman dimuat.

index.html

```
<!DOCTYPE html>
<html>
```

```
<head>
  <script src="karsa.js"></script>
</head>
<body>
  <script type="text/karsa">
    data nama = "Dunia"
    tampilkan pesan "Halo, " + nama
  </script>
</body>
</html>
```

Bab 2: Variabel dan Data

Variabel adalah wadah untuk menyimpan data. Dalam KARSA, terdapat tiga kata kunci untuk mendeklarasikan variabel, masing-masing dengan karakteristik reaktivitas dan mutabilitas yang berbeda. Memahami perbedaan ketiganya adalah fondasi utama untuk menguasai KARSA, karena pilihan jenis variabel akan mempengaruhi bagaimana data berinteraksi dengan tampilan dan komponen di aplikasimu.

2.1 data — Variabel Reaktif

Kata kunci **data** mendeklarasikan variabel reaktif. Variabel ini dibungkus dalam Proxy JavaScript sehingga setiap perubahan nilai akan secara otomatis memperbarui tampilan (DOM) tanpa perlu manipulasi manual. Ini adalah jenis variabel yang paling sering digunakan dalam KARSA karena model reaktivitas bawaan merupakan fitur inti bahasa ini. Variabel data bersifat writable, artinya nilainya bisa diubah menggunakan keyword **simpan**.

Contoh: Deklarasi dan perubahan data reaktif

```
data nama = "Budi"
data umur = 25
data aktif = benar

--! Mengubah nilai data reaktif
simpan "Andi" ke nama
simpan 30 ke umur
```

Di belakang layar, **data nama = "Budi"** dikompilasi menjadi **const nama = __createReactive("Budi")**. Proxy ini memiliki properti **.value** yang menyimpan nilai sebenarnya. Ketika kamu menulis **simpan "Andi" ke nama**, KARSA menghasilkan **__setState(nama, "Andi")** yang secara internal mengatur **nama.value = "Andi"** dan memicu semua subscriber yang bergantung pada variabel ini untuk memperbarui tampilan mereka.

*Catatan: Variabel **data** dikompilasi menggunakan **const** (bukan **let**) karena objek Proxy-nya sendiri tidak pernah di-assign ulang — hanya properti **.value**-nya yang berubah.*

2.2 tetap — Konstanta

Kata kunci **tetap** mendeklarasikan konstanta, yaitu variabel yang nilainya tidak dapat diubah setelah inisialisasi. Ini setara dengan **const** dalam JavaScript. Tetap berguna untuk nilai-nilai yang seharusnya tidak berubah sepanjang siklus hidup aplikasi, seperti konfigurasi, batas maksimum, URL API, dan sebagainya. Mencoba mengubah nilai tetap akan menghasilkan error E3003 dari Resolver.

Contoh: Deklarasi konstanta

```
tetap PI = 3.14159
tetap VERSI = "0.3.1"
tetap MAKS_PEMAIN = 10

--! Ini akan menghasilkan error E3003!
--! simpan 3.14 ke PI ← Error: variabel tetap tidak dapat diubah
```

Perhatian: Selalu berikan nilai awal untuk **tetap**. Deklarasi tanpa nilai awal akan memicu peringatan W4003 karena konstanta tanpa nilai tidak memiliki kegunaan praktis.

2.3 ubah — Variabel Mutable Non-Reaktif

Kata kunci **ubah** mendeklarasikan variabel yang bisa diubah (mutable) tetapi **tidak reaktif**. Ini berarti perubahan nilai variabel **ubah** tidak akan memperbarui tampilan secara otomatis. Variabel **ubah** setara dengan **let** dalam JavaScript. Gunakan **ubah** untuk variabel-variabel yang bersifat sementara atau internal, seperti counter loop, akumulator, atau variabel perhitungan yang tidak perlu ditampilkan langsung ke pengguna.

Contoh: Perbedaan simpan ke **ubah** vs **data**

```
ubah counter = 0
ubah hasil = 0

--! Mengubah nilai ubah langsung (bukan via __setState)
simpan 10 ke counter
--! Dikompilasi menjadi: counter = 10;

--! Bandingkan dengan data reaktif:
data nama = "Budi"
simpan "Andi" ke nama
--! Dikompilasi menjadi: __setState(nama, "Andi");
```

Perbedaan kompilasi ini sangat penting untuk dipahami. Saat kamu menulis **simpan 10 ke counter** dimana counter adalah variabel **ubah**, KARSA menghasilkan **counter = 10** (assignment langsung). Tetapi jika target adalah data reaktif, KARSA menghasilkan **__setState(nama, "Andi")** yang memicu pembaruan DOM. KARSA secara otomatis mendeteksi jenis variabel dari metadata yang dihasilkan Resolver dan memilih mekanisme assignment yang tepat.

2.4 Perbandingan Tiga Jenis Variabel

KARSA	JavaScript	Keterangan
<code>data nama = "x"</code>	<code>const nama = __createReactive("x")</code>	Reaktif, Proxy-based, perubahan memicu update DOM
<code>tetap PI = 3.14</code>	<code>const PI = 3.14</code>	Konstanta, tidak bisa diubah (E3003 jika ditulis ulang)
<code>ubah y = 10</code>	<code>let y = 10</code>	Mutable, non-reaktif, perubahan tidak memicu update DOM

Jenis	Karakteristik
data	Reaktif, writable, perubahan memicu subscriber update
tetap	Non-reaktif, read-only, error jika diassign ulang
ubah	Non-reaktif, writable, assignment langsung tanpa Proxy

2.5 Type Hint

KARSA mendukung type hint opsional untuk memberikan petunjuk tipe data yang diharapkan dari sebuah variabel. Type hint tidak bersifat wajib — variabel tanpa type hint akan tetap berfungsi normal. Namun, jika type hint diberikan dan nilai awal tidak sesuai, Analyzer akan menghasilkan peringatan W4001. Type hint ditulis setelah nama variabel dengan format **nama: tipe**.

Contoh: Type hint

```
data nama: teks = "Budi"
data umur: angka = 25
data aktif: benar-salah = benar
data daftar: array = [1, 2, 3]

--! Ini akan memicu peringatan W4001:
data harga: angka = "seribu" ← tipe teks, bukan angka!
```

Type Hint	Deskripsi dan Contoh
angka	Bilangan bulat atau desimal, contoh: 42, 3.14
teks	String teks, contoh: "halo", "KARSA"
benar-salah	Boolean, contoh: benar, salah
array	Daftar/lereng, contoh: [1, 2, 3]
objek	Objek, contoh: { nama: "Budi" }

Bab 3: Tipe Data

Setiap nilai dalam KARSA memiliki tipe data tertentu. Memahami tipe data adalah kunci untuk menulis program yang benar dan efisien. KARSA mengenali lima tipe data utama: angka, teks, benar-salah (boolean), array, dan objek. Meskipun KARSA tidak menerapkan type checking yang ketat (type hint bersifat opsional), pemahaman tentang tipe data akan membantumu menghindari bug logika yang umum terjadi.

3.1 Angka

Tipe angka merepresentasikan bilangan, baik bulat maupun desimal. Dalam KARSA, semua angka direpresentasikan sebagai Number JavaScript (IEEE 754 double-precision). Operasi aritmatika dasar seperti penjumlahan, pengurangan, perkalian, dan pembagian dapat dilakukan langsung pada tipe angka. Perhatikan bahwa operasi pembagian dengan nol akan menghasilkan **Infinity** atau **NaN**, sama seperti di JavaScript.

Contoh: Tipe angka

```
data umur = 25
data harga = 15000.50
data negatif = -10

--! Operasi aritmatika
data total = harga kali 2
data sisa = 10 mod 3 --! hasilnya 1
data kuadrat = 5 pangkat 2 --! hasilnya 25
```

3.2 Teks

Tipe teks merepresentasikan deretan karakter (string). Teks dalam KARSA ditulis dengan tanda kutip ganda "...". Penggabungan teks dilakukan dengan operator **+**, sama seperti di JavaScript. KARSA juga mendukung akses properti string seperti **.panjang** (alias untuk **.length**) untuk mendapatkan panjang teks.

Contoh: Tipe teks

```
data nama = "Budi Santoso"
data sapaan = "Halo, " + nama
data panjang = nama.panjang --! alias untuk .length

--! Template literal belum didukung, gunakan penggabungan +
data info = "Nama: " + nama + ", umur: " + 25
```

Catatan: Properti **.panjang** adalah alias Indonesia untuk **.length**. KARSA secara otomatis menerjemahkan alias properti Indonesia ke properti JavaScript yang sesuai melalui tabel **ALIAS_PROPERTI** di Resolver.

3.3 Benar-Salah (Boolean)

Tipe benar-salah merepresentasikan nilai kebenaran. KARSA menggunakan kata kunci **benar** untuk true dan **salah** untuk false. Tipe ini terutama digunakan dalam kondisi perbandingan dan pengambilan keputusan. Operator logika seperti **dan**, **atau**, dan **bukan** beroperasi pada tipe benar-salah.

Contoh: Tipe benar-salah

```
data aktif = benar
data selesai = salah

--! Hasil operasi perbandingan adalah benar-salah
data dewasa = umur lebih dari 17
data kosong = daftar.panjang sama dengan 0

--! Operator logika
data bolehMasuk = dewasa dan aktif
data tidakAktif = bukan aktif
```

3.4 Array

Array adalah kumpulan data terurut yang disimpan dalam satu variabel. Dalam KARSA, array dideklarasikan dengan tanda kurung siku [...]. Elemen array diakses menggunakan indeks berbasis nol. KARSA menyediakan keyword **sisipkan** untuk menambahkan elemen ke array (setara dengan `.push()` di JavaScript) dan **ulangi** untuk iterasi over array.

Contoh: Array

```
data buah = ["Apel", "Mangga", "Jeruk"]
data angka = [10, 20, 30, 40, 50]

--! Akses elemen berdasarkan indeks
data pertama = buah[0] --! "Apel"

--! Tambah elemen baru
sisipkan "Pisang" ke buah

--! Iterasi array
ulangi item dari buah:
    tampilkan pesan item
```

3.5 Objek

Objek adalah kumpulan pasangan kunci-nilai (key-value pairs). Objek digunakan untuk merepresentasikan entitas yang memiliki beberapa properti. Dalam KARSA, objek dideklarasikan dengan kurung kurawal {...} dan akses properti menggunakan notasi titik. KARSA mendukung alias properti Indonesia untuk properti DOM umum, seperti **.teks** untuk **.innerText** dan **.html** untuk **.innerHTML**.

Contoh: Objek

```
data siswa = { nama: "Budi", umur: 17, kelas: "XII" }

--! Akses properti
data namaSiswa = siswa.nama

--! Objek bersarang (nested)
data sekolah = {
    nama: "SMAN 1",
    alamat: { kota: "Jakarta", provinsi: "DKI" }
}
```

Bab 4: Operator

Operator adalah simbol atau kata kunci yang melakukan operasi terhadap satu atau lebih nilai (operand). KARSA menyediakan tiga kategori operator utama: aritmatika untuk perhitungan matematis, perbandingan untuk membandingkan nilai, dan logika untuk menggabungkan atau membalik kondisi boolean. Semua operator KARSA menggunakan kata kunci berbahasa Indonesia yang intuitif dan mudah diingat.

4.1 Operator Aritmatika

Operator aritmatika digunakan untuk melakukan perhitungan matematis. KARSA menyediakan enam operator aritmatika dasar, ditambah operator pangkat untuk perpangkatan. Semua operator ini menghasilkan nilai bertipe angka. Perhatikan bahwa operator menggunakan kata Indonesia, bukan simbol matematika konvensional, untuk menjaga konsistensi dengan filosofi bahasa KARSA.

KARSA	JavaScript	Keterangan
tambah	+	Penjumlahan: 5 tambah 3 = 8
kurang	-	Pengurangan: 10 kurang 4 = 6
kali	*	Perkalian: 6 kali 7 = 42
bagi	/	Pembagian: 20 bagi 4 = 5
mod	%	Sisa bagi: 10 mod 3 = 1
pangkat	**	Perpangkatan: 2 pangkat 3 = 8

Contoh: Operator aritmatika

```
data a = 10
data b = 3

data jumlah = a tambah b --! 13
data selisih = a kurang b --! 7
data kali = a kali b --! 30
data bagi = a bagi b --! 3.333...
data sisa = a mod b --! 1
data kuadrat = a pangkat 2 --! 100
```

Catatan: Selain operator aritmatika, KARSA menyediakan keyword **tambahkan** dan **kurangi** untuk mutasi variabel secara ringkas. Contoh: **tambahkan 5 ke counter** setara dengan **simpan counter + 5 ke counter** untuk variabel **ubah**, atau **__setState(counter, counter.value + 5)** untuk data reaktif.

4.2 Operator Perbandingan

Operator perbandingan membandingkan dua nilai dan menghasilkan tipe benar-salah. KARSA menggunakan strict equality (=== dalam JavaScript) secara default, artinya baik nilai maupun tipe data harus sama. Operator perbandingan sering digunakan dalam kondisi **jika** dan **selama**.

KARSA	JavaScript	Keterangan
sama dengan	===	Sama persis (nilai dan tipe)
tidak sama dengan	!==	Tidak sama (nilai atau tipe berbeda)
lebih dari	>	Lebih besar
kurang dari	<	Lebih kecil
paling sedikit	>=	Lebih besar atau sama dengan
paling banyak	<=	Lebih kecil atau sama dengan

Contoh: Operator perbandingan

```
data umur = 18

data dewasa = umur paling sedikit 17 --! benar
data anak = umur kurang dari 17 --! salah
data tepat = umur sama dengan 18 --! benar
data bukan17 = umur tidak sama dengan 17 --! benar
```

4.3 Operator Logika

Operator logika menggabungkan atau membalik nilai benar-salah. KARSA menyediakan tiga operator logika yang setara dengan operator JavaScript. Operator **dan** menghasilkan benar hanya jika kedua operand benar. Operator **atau** menghasilkan benar jika salah satu operand benar. Operator **bukan** membalik nilai boolean dari benar menjadi salah dan sebaliknya.

KARSA	JavaScript	Keterangan
dan	&&	Logika DAN: benar dan benar = benar
atau		Logika ATAU: benar atau salah = benar
bukan	!	Negasi: bukan benar = salah

Contoh: Operator logika

```
data umur = 20
data punyaKTP = benar

--! Kombinasi operator perbandingan dan logika
data bolehVoting = umur paling sedikit 17 dan punyaKTP
--! hasilnya: benar

data butuhPendamping = umur kurang dari 17 atau bukan punyaKTP
--! hasilnya: salah

--! Negasi
data belumDewasa = bukan (umur paling sedikit 17)
--! hasilnya: salah
```

Bab 5: Output — tampilkan

Dalam pemrograman web, "output" berarti menampilkan sesuatu ke halaman web (DOM). KARSA menyediakan keyword **tampilkan** untuk menampilkan konten ke halaman. Ada tiga bentuk utama: **tampilkan pesan** `"..."` untuk pesan biasa, **tampilkan pesan-error** `"..."` untuk pesan error, dan **tampilkan notifikasi** `"..."` untuk notifikasi. Berbeda dengan `jalankan console.log()` yang hanya menampilkan output di konsol browser, **tampilkan** secara langsung memanipulasi DOM sehingga pengguna dapat melihat hasilnya di halaman web.

5.1 Tampilkan Dasar

Penggunaan paling sederhana dari **tampilkan** adalah menampilkan pesan teks atau nilai variabel ke dalam halaman. KARSA menyediakan tiga jenis pesan: **pesan** (pesan biasa), **pesan-error** (pesan error), dan **notifikasi** (notifikasi). Selain itu, **tampilkan** juga bisa digunakan untuk menampilkan elemen DOM dengan selector.

Contoh: Tampilkan dasar

```
--! Menampilkan teks langsung
tampilkan pesan "Halo, dunia!"

--! Menampilkan nilai variabel
data nama = "Budi"
tampilkan pesan "Halo, " + nama

--! Menampilkan hasil ekspresi
data a = 10
data b = 20
tampilkan pesan "Jumlah: " + (a tambah b)
```

5.2 Mode Tampilkan

KARSA menyediakan beberapa mode **tampilkan** untuk mengontrol bagaimana konten ditambahkan ke DOM. Mode ini menentukan apakah konten baru menggantikan konten lama, ditambahkan setelahnya, atau disisipkan sebelumnya. Pemilihan mode yang tepat sangat penting untuk membangun antarmuka yang dinamis dan responsif.

Sintaks	Deskripsi
<code>tampilkan pesan "..."</code>	Menampilkan pesan teks ke DOM (mode default: tambahkan)
<code>tampilkan pesan "..."</code> dengan mode: <code>"ganti"</code>	Mengganti seluruh konten elemen dengan konten baru
<code>tampilkan pesan "..."</code> dengan mode: <code>"awalan"</code>	Menyisipkan konten di awal elemen
<code>tampilkan pesan "..."</code> dengan mode: <code>"sebelum"</code>	Menyisipkan konten sebelum elemen target

Sintaks	Deskripsi
tampilkan pesan "... " dengan mode: "sesudah"	Menyisipkan konten setelah elemen target

Contoh: Mode tampilkan

```
--! Mode default (tambahkan)
tampilkan pesan "Baris pertama"
tampilkan pesan "Baris kedua" --! ditambahkan setelah baris pertama

--! Mode ganti - mengganti konten sebelumnya
tampilkan pesan "Konten baru" dengan mode: "ganti"

--! Mode awalan - menyisipkan di awal
tampilkan pesan "Header" dengan mode: "awalan"
```

Catatan: Untuk kebutuhan debugging, gunakan jalankan `console.log()` untuk menampilkan output di konsol browser. Ini berguna untuk melihat nilai variabel tanpa mempengaruhi tampilan halaman.

Bab 6: Proyek Mini — Kalkulator Sederhana

Sekarang saatnya menggabungkan semua konsep yang telah kita pelajari ke dalam satu proyek nyata! Kita akan membuat kalkulator sederhana yang bisa melakukan operasi penjumlahan, pengurangan, perkalian, dan pembagian. Proyek ini akan menggunakan variabel, tipe data, operator, dan tampilan.

Langkah 1: Siapkan Variabel

Pertama, kita mendeklarasikan variabel-variabel yang dibutuhkan. Kita akan menggunakan dua variabel reaktif untuk angka input, satu variabel reaktif untuk hasil, dan satu variabel ubah untuk menyimpan operator yang dipilih.

Langkah 1: Deklarasi variabel

```
--! Variabel untuk input
data angka1 = 0
data angka2 = 0

--! Variabel untuk hasil
data hasil = 0

--! Operator aktif (gunakan ubah karena tidak perlu reaktif)
ubah operator = "tambah"
```

Langkah 2: Buat Fungsi Hitung

Selanjutnya, kita buat fungsi yang menghitung hasil berdasarkan operator yang dipilih. Fungsi ini akan membaca nilai dari angka1 dan angka2, melakukan perhitungan, dan menyimpan hasilnya ke variabel hasil.

Langkah 2: Fungsi hitung

```
fungsi hitung():
  jika operator sama dengan "tambah":
    simpan angka1 tambah angka2 ke hasil
  kalau operator sama dengan "kurang":
    simpan angka1 kurang angka2 ke hasil
  kalau operator sama dengan "kali":
    simpan angka1 kali angka2 ke hasil
  kalau operator sama dengan "bagi":
    jika angka2 sama dengan 0:
      tampilkan pesan-error "Error: Tidak bisa bagi nol!"
    jika tidak:
      simpan angka1 bagi angka2 ke hasil
```

Langkah 3: Tampilkan Hasil

Terakhir, kita gunakan tampilkan untuk menampilkan hasil perhitungan ke pengguna. Karena hasil adalah variabel reaktif (data), setiap kali nilainya berubah, tampilan akan otomatis diperbarui.

Langkah 3: Penggunaan dan tampilan

```
--! Contoh penggunaan
simpan 15 ke angka1
simpan 5 ke angka2
simpan "kali" ke operator

hitung()

--! Tampilkan hasil
tampilkan pesan "Kalkulator KARSA"
tampilkan pesan "Angka 1: " + angka1
tampilkan pesan "Angka 2: " + angka2
tampilkan pesan "Operator: " + operator
tampilkan pesan "Hasil: " + hasil
```

Output yang Diharapkan

Setelah kompilasi dan eksekusi, program di atas akan menghasilkan tampilan seperti berikut di halaman web:

Output

```
Kalkulator KARSA
Angka 1: 15
Angka 2: 5
Operator: kali
Hasil: 75
```

Catatan: Ini adalah contoh sederhana. Di modul selanjutnya, kamu akan belajar cara membuat antarmuka interaktif dengan **buat** dan **ketika** sehingga pengguna bisa mengklik tombol untuk memasukkan angka dan memilih operator.

Latihan Soal

Kerjakan latihan berikut untuk menguji pemahamanmu tentang materi di modul ini. Soal-soal disusun dari tingkat kesulitan mudah hingga sulit. Untuk soal praktik, tuliskan kode KARSA yang sesuai. Solusi lengkap tersedia di bagian akhir modul.

Soal Mudah (Teori)

1. Jelaskan perbedaan utama antara **data**, **tetap**, dan **ubah** dalam KARSA. Berikan satu contoh penggunaan yang tepat untuk masing-masing jenis variabel.
2. Apa yang dimaksud dengan "reaktif" dalam konteks KARSA? Mengapa variabel **data** bersifat reaktif sementara **ubah** tidak?
3. Sebutkan lima tahap pipeline KARSA dan jelaskan secara singkat fungsi masing-masing tahap. Jika kamu menulis **data x = 10**, apa yang terjadi di setiap tahap?
4. Tuliskan operator KARSA yang setara dengan operator JavaScript berikut: `===`, `>=`, `&&`, `!`, `**`.

Soal Sedang (Praktik)

5. Tulis kode KARSA untuk mendeklarasikan variabel-variabel berikut: (a) variabel reaktif "nama" dengan nilai "KARSA", (b) konstanta "VERSI" dengan nilai "0.3.1", (c) variabel mutable "counter" dengan nilai awal 0, kemudian ubah nilai counter menjadi 10.
6. Tulis kode KARSA yang mendeklarasikan array berisi 5 nama buah, kemudian tambahkan satu buah baru menggunakan **sisipkan**, dan tampilkan jumlah elemen array menggunakan **.panjang**.
7. Tulis kode KARSA yang menghitung luas segitiga ($\text{alas} \times \text{tinggi} / 2$) menggunakan variabel **data** untuk alas dan tinggi, lalu tampilkan hasilnya menggunakan **tampilkan**.

Soal Sulit (Analisis dan Debug)

8. Perhatikan kode KARSA berikut. Temukan dan jelaskan minimal dua error yang akan terjadi saat kompilasi, termasuk kode error-nya:
9. Bandingkan output JavaScript yang dihasilkan oleh dua kode KARSA berikut dan jelaskan mengapa hasilnya berbeda:

Soal 8

```
--! Kode untuk soal nomor 8
tetap nama = "KARSA"
simpan "JavaScript" ke nama
ubah y = 10
data z: angka = "bukan angka"
```

Soal 9

```
--! Kode A:
data x = 10
simpan 20 ke x

--! Kode B:
ubah x = 10
simpan 20 ke x
```

10. Tulis program KARSA lengkap yang membuat konverter suhu sederhana: mendeklarasikan variabel suhu Celcius (data), menghitung suhu Fahrenheit menggunakan rumus $F = C * 9/5 + 32$, dan menampilkan kedua nilai suhu. Gunakan keyword yang sesuai untuk setiap variabel berdasarkan karakteristiknya.

Solusi Latihan

Solusi Soal Mudah

1. **data** mendeklarasikan variabel reaktif yang dibungkus Proxy. Perubahan nilainya otomatis memperbarui DOM. Contoh: **data nama = "Budi"** — digunakan untuk data yang ditampilkan di tampilan. **tetap** mendeklarasikan konstanta yang tidak bisa diubah setelah inisialisasi. Contoh: **tetap PI = 3.14** — digunakan untuk nilai tetap. **ubah** mendeklarasikan variabel mutable non-reaktif. Perubahan nilai tidak memicu update DOM. Contoh: **ubah counter = 0** — digunakan untuk variabel internal yang tidak ditampilkan.

2. "Reaktif" berarti bahwa perubahan nilai variabel secara otomatis memicu pembaruan tampilan. Variabel **data** bersifat reaktif karena dikompilasi menjadi **const x = __createReactive(nilai)** yang membungkus nilai dalam Proxy. Proxy ini mengintercept penugasan dan memberitahu semua subscriber. Variabel **ubah** tidak reaktif karena dikompilasi menjadi **let x = nilai** — assignment biasa tanpa mekanisme notifikasi.

3. Lima tahap: (1) Lexer — memecah kode menjadi token; (2) Parser — membangun AST dari token; (3) Resolver — mengikat identifier ke deklarasi dan deteksi duplikat; (4) Analyzer — validasi semantik; (5) Compiler — menghasilkan JavaScript. Untuk **data x = 10**: Lexer menghasilkan [data, x, =, 10], Parser membuat node DataDeclaration, Resolver mendaftarkan "x" sebagai simbol reaktif, Analyzer memvalidasi type hint jika ada, Compiler menghasilkan **const x = __createReactive(10)**.

4. Operator KARSA yang setara: **===** → sama dengan, **>=** → paling sedikit, **&&** → dan, **!** → bukan, ****** → pangkat.

Solusi Soal Sedang

5.

```
data nama = "KARSA"
tetap VERSI = "0.3.1"
ubah counter = 0
simpan 10 ke counter
```

6.

```
data buah = ["Mangga", "Apel", "Jeruk", "Pisang", "Durian"]
sisipkan "Rambutan" ke buah
tampilkan pesan "Jumlah buah: " + buah.panjang
```

7.

```
data alas = 10
data tinggi = 8
data luas = (alas kali tinggi) bagi 2
tampilkan pesan "Luas segitiga: " + luas
```

Solusi Soal Sulit

8. Dua error yang akan terjadi: (1) Error E3003 — mencoba mengubah variabel **tetap** "nama" menggunakan **simpan**. Variabel tetap bersifat read-only. (2) Peringatan W4001 — type hint **angka** untuk variabel **z** tidak cocok dengan nilai awal bertipe teks ("bukan angka").

9. Kode A menggunakan **data x = 10; simpan 20 ke x** dan menghasilkan **__setState(x, 20)** — perubahan melalui fungsi reaktif yang memicu subscriber. Kode B menggunakan **ubah x = 10; simpan 20 ke x** dan menghasilkan **x = 20** — assignment langsung tanpa Proxy. Perbedaan utama: Kode A memicu update DOM otomatis, Kode B tidak.

10.

```
--! Konverter suhu sederhana
data celcius = 25
tetap FAKTOR = 9 bagi 5 --! 1.8
tetap PENAMBAH = 32

data fahrenheit = (celcius kali FAKTOR) tambah PENAMBAH

tampilkan pesan "Suhu dalam Celcius: " + celcius
tampilkan pesan "Suhu dalam Fahrenheit: " + fahrenheit

--! Coba ubah suhu dan lihat update otomatis!
simpan 100 ke celcius
--! Fahrenheit akan otomatis berubah karena data reaktif
```

Lampiran: Referensi Cepat KARSA vs JS

Tabel berikut menyediakan referensi cepat perbandingan sintaks antara KARSA dan JavaScript. Gunakan tabel ini sebagai cheat sheet saat belajar atau menulis kode KARSA.

Deklarasi Variabel

KARSA	JavaScript	Keterangan
data x = 10	const x = __createReactive(10)	Variabel reaktif
tetap PI = 3.14	const PI = 3.14	Konstanta
ubah y = 5	let y = 5	Variabel mutable

Assignment dan Mutasi

KARSA	JavaScript	Keterangan
simpan 20 ke x	__setState(x, 20) / x = 20	Assignment (reaktif/non-reaktif)
tambahkan 5 ke x	__setState(x, x.value+5) / x+=5	Increment
kurangi x dengan 3	__setState(x, x.value-3) / x-=3	Decrement
sisipkan "a" ke arr	arr.value.push("a") / arr.push("a")	Push ke array

Operator

KARSA	JavaScript	Keterangan
tambah / kurang / kali / bagi	+ / - / * / /	Aritmatika dasar
mod / pangkat	% / **	Sisa bagi dan perpangkatan
sama dengan / tidak sama dengan	=== / !==	Perbandingan ketat
lebih dari / kurang dari	> / <	Perbandingan
paling sedikit / paling banyak	>= / <=	Perbandingan dengan kesamaan
dan / atau / bukan	&& / / !	Logika boolean

Kontrol Alur

KARSA	JavaScript	Keterangan
jika ... : ... kalau ... : ... jika tidak: ...	if (...) { } else if (...) { } else { }	Pengkondisian
selama kondisi:	while (kondisi) { }	Loop dengan kondisi

KARSA	JavaScript	Keterangan
ulangi item dari array:	<code>for (let item of array) { }</code>	Iterasi array
fungsi nama():	<code>function nama() { }</code>	Deklarasi fungsi
kembalikan nilai	<code>return nilai</code>	Return dari fungsi
berhenti / lewati	<code>break / continue</code>	Kontrol loop

DOM dan Output

KARSA	JavaScript	Keterangan
tampilkan pesan "halo"	<code>element.innerText = "halo"</code>	Tampilkan pesan ke DOM
buat tombol "Klik":	<code>document.createElement("button")</code>	Buat elemen DOM
ketika diklik:	<code>element.addEventListener("click")</code>	Event handler
perbarui teks elemen	<code>element.innerText = ...</code>	Update properti DOM
jalankan <code>console.log()</code>	<code>console.log()</code>	JS interop

Alias Properti Indonesia

KARSA (Alias)	JavaScript (Asli)
<code>.panjang</code>	<code>.length</code>
<code>.nilai</code>	<code>.value</code>
<code>.teks</code>	<code>.innerText</code>
<code>.html</code>	<code>.innerHTML</code>
<code>.tipe</code>	<code>.type</code>
<code>.nama</code>	<code>.name</code>
<code>.ditandai</code>	<code>.checked</code>
<code>.nonaktif</code>	<code>.disabled</code>
<code>.anak</code>	<code>.children</code>
<code>.induk</code>	<code>.parentElement</code>
<code>.fokus</code>	<code>.focus</code>
<code>.atribut</code>	<code>.getAttribute</code>